

Computer Organization and Architecture Laboratory

MiniRISC Assignment 1B: ALU, Register File, Basic Datapath and Instruction BRAM

Indian Institute of Technology Kharagpur
Department of Computer Science and Engineering

Deadline: 7 October 2026

Objective

Using the processor architecture and ISA planned in Assignment 1A, implement the **baseline computational datapath** of the 32-bit MiniRISC processor and connect it to a **BRAM-based instruction memory**.

The work for this assignment includes:

- the 32-bit ALU;
- the 16×32 register file;
- immediate extraction and extension;
- ALU-input selection;
- ALU-result write-back;
- a basic Program Counter and instruction-fetch path using BRAM;
- simple instruction-field extraction / decoding;
- behavioral simulation of the integrated datapath.

The Booth multiplier, MUL/MAC execution hardware, multi-cycle multiplier control, and the complete final processor control FSM are not part of this assignment.

1. Mandatory ALU Functions

The 32-bit ALU must support at least the following operations:

Operation	Typical Mnemonic	Function
Addition	ADD	$Y = A + B$
Subtraction	SUB	$Y = A - B$
Bitwise AND	AND	$Y = A \wedge B$
Bitwise OR	OR	$Y = A \vee B$
Bitwise XOR	XOR	$Y = A \oplus B$
Bitwise NOR	NOR	$Y = \neg(A \vee B)$
Bitwise NOT	NOT	$Y = \neg A$
Logical left shift	SL/SLL	$Y = A \ll \text{shamt}$
Logical right shift	SRL	$Y = A \gg \text{shamt}$
Arithmetic right shift	SRA	signed right shift of A
Set-less-than	SLT	$Y = 1$ if signed $A < B$, else 0
Set-greater-than	SGT	$Y = 1$ if signed $A > B$, else 0

Immediate forms such as ADDI and SUBI should reuse the same ALU hardware by selecting an immediate value instead of a second register operand.

Example:

ADDI R3, R2, #25

means

$$R3 \leftarrow R2 + 25.$$

The ALU still performs an ordinary addition. The only difference is that its second input comes from the immediate field of the instruction rather than from a second source register.

Groups may implement additional ALU functions required by their planned ISA.

2. Register File

Implement a register file containing

16 registers $\times$ 32 bits.

The register file must provide:

- two independent read ports;
- one write port;
- two read-register addresses;
- one write-register address;
- one write-enable signal;
- synchronous write operation;
- $R0$ permanently equal to zero.

Two read ports mean that two source registers can be read at the same time.

For example,

ADD R5, R2, R3

requires the register file to supply

$R2$ and $R3$

simultaneously to the ALU.

The ALU result is then written back into $R5$ through the single write port.

Any attempted write to $R0$ must be ignored.

3. Immediate Extraction and Extension Logic

Some instructions contain a constant value or offset directly inside the instruction word. This value is called an **immediate**.

The processor must extract this immediate field and convert it into a 32-bit value before it is used by the datapath.

For example,

ADDI R3, R2, #25

contains the constant 25 inside the instruction. The immediate logic must extract that field and provide a 32-bit value to the ALU.

For a signed arithmetic immediate, **sign extension** is normally required.

For example, if an 8-bit immediate field contains

11111111_2 ,

which represents -1 , then its 32-bit sign-extended form is

11111111 11111111 11111111 11111111.

For some logical immediate instructions, **zero extension** may be appropriate.

For example,

10101100_2

may be extended to

00000000 00000000 00000000 10101100.

The same idea is also used for memory offsets.

For example,

LD R4, 8(R5)

requires the processor to form the effective address

$$EA = R5 + 8.$$

Similarly,

ST R6, -4(R7)

requires

$$EA = R7 - 4.$$

Hence, the block may be implemented as a separate module or as part of the instruction decoder. A separate module is **not mandatory**.

4. ALU-Input Selection Multiplexer

The second ALU operand may come from different sources.

For a register-register instruction,

ADD R5, R2, R3

the ALU inputs are

$$A = R2, \quad B = R3.$$

For an immediate instruction,

ADDI R5, R2, #10

the ALU inputs are

$$A = R2, \quad B = 10.$$

Therefore, a multiplexer is required before the second ALU input.

Conceptually,

$\text{ALU Input B} = \begin{cases} \text{Register Read Data 2,} & \text{register-register instruction} \\ \text{Extended Immediate,} & \text{immediate instruction} \end{cases}$

The select signal for this multiplexer is generated from the decoded instruction.

5. Program Counter and Instruction Fetch

The Program Counter (PC) identifies the current instruction to be fetched.

For normal sequential execution, the PC advances to the next instruction.

If the instruction memory is organized as one 32-bit instruction per address, a simple convention is

$$PC \leftarrow PC + 1.$$

If a group chooses byte-addressed instruction memory, the group may instead use a different PC increment such as

$$PC \leftarrow PC + 4.$$

The chosen convention must match the group's ISA and BRAM organization and must be documented.

At the block level, instruction fetch is:

$$\boxed{PC \longrightarrow \text{Instruction BRAM} \longrightarrow \text{32-bit Instruction}}$$

The instruction word is then divided into fields such as:

- opcode;
- destination-register field;
- source-register fields;
- immediate field.

The exact bit positions depend on the instruction format chosen by the group in Assignment 1A.

6. Instruction Decoder / Field Extraction

The instruction decoder examines the fetched instruction and determines what operation is requested.

For example, for an instruction such as

ADD R5, R2, R3

the decoder should identify:

- the opcode corresponding to ADD;
- destination register *R5*;
- source register *R2*;
- source register *R3*;

- the required ALU operation;
- that the second ALU input comes from a register;
- that the ALU result should be written back into the register file.

For

```
ADDI R5, R2, #10
```

the decoder should identify that the second ALU operand must come from the immediate-extension logic.

For Assignment 1B, a **minimal decoder sufficient for the demonstrated instructions** is acceptable. The complete final control FSM is not yet required.

7. Mandatory BRAM-Based Instruction Memory

The instruction memory must be implemented using FPGA **Block RAM (BRAM)**.

A register array implemented using ordinary Verilog registers is **not permitted** for the instruction memory.

The BRAM must be configured as an instruction **ROM**. Depending on the processor architecture, students may use:

- a **Single-Port ROM**, or
- a **Dual-Port ROM**.

7.1 What is a Single-Port Instruction ROM?

A single-port ROM provides one address input and one instruction output.

Conceptually,

Address → ROM → Instruction

This is sufficient for a simple processor that fetches only one instruction at a time.

7.2 What is a Dual-Port Instruction ROM?

A dual-port ROM provides two independently accessible ports to the same BRAM contents.

A dual-port ROM is only necessary if the chosen architecture genuinely requires two simultaneous instruction-memory accesses.

For a simple MiniRISC implementation, a **single-port instruction ROM is normally sufficient**.

Students should not use a dual-port ROM unless they can explain why it is useful in their architecture.

7.3 Initialization of the Instruction ROM

The instruction ROM should be initialized with a small test program using the FPGA tool's memory-initialization mechanism.

For Vivado Block Memory Generator, this may be done using a `.coe` file.

For example, if the group wants the processor to execute:

```
ADDI R1, R0, #5
ADDI R2, R0, #7
ADD  R3, R1, R2
```

the corresponding 32-bit machine-code words should be placed in the instruction BRAM initialization file.

The exact hexadecimal or binary words depend on the group's ISA encoding.

7.4 BRAM Read Timing

Students must also understand that FPGA BRAM is normally **synchronous memory**.

Depending on the selected BRAM configuration, the instruction corresponding to an address may become available only after an active clock edge.

Therefore, students must check the timing behavior of the configured BRAM IP and ensure that the PC, instruction fetch, and instruction execution are sequenced consistently.

For example, if the BRAM has one-cycle read latency:

Cycle n : *PC* supplies the address

and

Cycle $n + 1$: the corresponding instruction becomes available.

The implementation must account for the actual BRAM configuration selected in Vivado.

8. Data Memory Requirement

If the group's implementation or test program requires data-memory access, the data memory must also be implemented using **BRAM**.

A register-array implementation of the data memory is not permitted.

The data memory may be configured as:

- a **Single-Port RAM**, or
- a **Dual-Port RAM**.

8.1 Single-Port Data RAM

A single-port RAM allows one memory operation through one port at a time.

This is sufficient if the processor performs at most one data-memory read or write in a cycle.

For example,

```
LD R4, 8(R5)
```

may calculate

$$EA = R5 + 8$$

and then use that address for one RAM read.

Similarly,

```
ST R6, -4(R7)
```

performs one RAM write.

8.2 Dual-Port Data RAM

A dual-port RAM allows two ports to access the same memory.

This may be useful if the chosen processor architecture requires two simultaneous memory accesses.

For the basic MiniRISC processor, a single-port data RAM is generally sufficient unless the group has a specific reason for requiring two ports.

The choice must be explained during the demonstration.

If data memory is not required for the Assignment 1B demonstration, its complete implementation may be postponed. However, once data memory is introduced, it must use BRAM.

9. Write-Back Path

After an operation is completed, the result must be returned to the register file.

For an arithmetic instruction such as

```
ADD R5, R2, R3
```

the write-back value is the ALU result:

$$R5 \leftarrow ALU_result.$$

Later, for a load instruction such as

```
LD R5, 8(R2)
```

the value written into *R5* will come from data memory rather than directly from the ALU.

Therefore, the full processor will eventually require a write-back multiplexer such as

$$\text{Register Write Data} = \begin{cases} \text{ALU Result,} & \text{arithmetic/logical instruction} \\ \text{Memory Read Data,} & \text{load instruction} \\ \text{Other Result,} & \text{future extension} \end{cases}$$

For Assignment 1B, the ALU-result path must be implemented. The memory-result path may be included if data memory is already being used.

10. Control Signals

A control signal is simply a signal that selects what the datapath should do.

Typical control signals include:

- ALU operation select;
- register write enable;
- ALU source select;
- destination-register select;
- memory read enable;
- memory write enable;
- write-back select;
- PC enable / PC select.

For example, for

ADD R5, R2, R3

the important control choices are conceptually:

- read *R2* and *R3*;
- select register data as both ALU operands;
- select ADD as the ALU operation;
- enable register write;
- select *R5* as destination;
- select the ALU result for write-back.

The final processor will generate these signals through a complete control unit / FSM. For Assignment 1B, only the control required for the demonstrated basic instructions is necessary.

11. Basic Datapath Integration

The integrated datapath should support operations of the form

$$R_d \leftarrow R_{s1} \text{ op } R_{s2}$$

and

$$R_d \leftarrow R_{s1} \text{ op } \textit{Immediate}.$$

A useful conceptual data flow is:

$$PC \rightarrow \textit{Instruction BRAM} \rightarrow \textit{Decode} \rightarrow \textit{Register File} \rightarrow \textit{ALU} \rightarrow \textit{Write Back}$$

For an immediate instruction:

$$\textit{Instruction} \rightarrow \textit{Immediate Extraction/Extension} \rightarrow \textit{ALU Input MUX} \rightarrow \textit{ALU}$$

The design must be consistent with the architecture and ISA submitted in Assignment 1A.

12. Minimum Behavioral Simulation

The behavioral testbench must demonstrate at least:

1. correct fetching of at least three different instructions from BRAM;
2. ADD using two source registers;
3. SUB using two source registers;
4. one logical operation;
5. one shift operation;
6. one signed comparison operation;
7. one immediate arithmetic operation;
8. correct register write-back;
9. simultaneous reading of two source registers;
10. attempted write to $R0$, showing that $R0$ remains zero.

The waveform should show, where applicable:

- clock;
- reset;
- PC / instruction address;
- fetched instruction;

- decoded register addresses;
- register read values;
- immediate value;
- ALU-source selection;
- ALU-control value;
- ALU operands;
- ALU result;
- destination-register address;
- register write enable;
- register write-back data.

13. Suggested Module Organization

A possible hierarchy is:

```
minirisc_basic_top
|
|-- pc_unit
|-- instruction_bram_rom
|-- instruction_decoder
|-- register_file_16x32
|-- immediate_extension_logic
|-- alu_input_mux
|-- alu32
'-- writeback_logic
```

If data memory is already required:

```
|-- data_bram_ram
```

Students may choose a different module decomposition if the same functionality is clearly present.

14. Explicitly Not Required by 7 October

The following are intentionally deferred:

- Booth multiplier integration;
- MUL execution hardware;
- MAC execution hardware;
- multi-cycle multiplier control;
- multiplier stall logic;

- NOP scheduling for multiplier latency;
- complete final branch/control FSM;
- matrix multiplication;
- image convolution;
- UART output.

15. Submission and Demonstration

Each group must submit:

- all Verilog source files;
- behavioral testbench;
- simulation waveforms;
- BRAM instruction-memory configuration;
- instruction-memory initialization file, e.g. `.coe`;
- data-memory BRAM configuration and initialization file, if data memory is used;
- updated architecture diagram, if modified after Assignment 1A;
- a short note describing the datapath and module hierarchy.

During the demonstration, the TA may ask any group member to explain:

- how the PC selects the next instruction;
- how the instruction BRAM is organized;
- why single-port or dual-port BRAM was selected;
- how instruction fields are decoded;
- how immediate values are extracted and extended;
- how the ALU chooses between register and immediate operands;
- how results are written back;
- how *R0* is protected from writes;